

## **Q7a) Explain Types of Arrays in Ruby? Explain the Array Operations in Ruby.**

**Arrays in Ruby** - An array in Ruby is a collection used to store multiple elements in a single variable. Arrays can store: Numbers, Strings, Objects, Mixed data types

**Creating an Array in Ruby** - `arr = [10,20,30,40]`

**Types of Arrays in Ruby** -

**1) Single Dimensional Array** - Stores elements in a single row.

**Example** - `arr = [10,20,30]`

```
puts arr[0]
```

```
puts arr[1]
```

**Output** - 10

20

**2) Multidimensional Array** - Array containing another array.

**Example** - `arr = [[1,2],[3,4]]`

```
puts arr[0][1]
```

```
puts arr[1][0]
```

**Output** - 2

3

**3) Mixed Array** - Stores different data types together.

**Example** - `arr = [10,"Rahul",3.14,true]`

```
puts arr[1]
```

**Output** - Rahul

**Array Operations in Ruby** -

**1) Accessing Elements** - Elements are accessed using index.

**Example** - `arr = [10,20,30]`

```
puts arr[2]
```

**Output** - 30

**2) Adding Elements - Using push()**

```
arr = [1,2]
```

```
arr.push(3)
```

```
puts arr
```

**Output** - 1 2 3

**3) Removing Elements - Using pop()**

```
arr = [1,2,3]
arr.pop
puts arr
```

**Output -** 1  
2

#### **4) Array Length - Using length**

```
arr = [10,20,30]
puts arr.length
```

**Output -** 3

#### **5) Iterating Array - Using each**

```
arr = [10,20,30]
arr.each do |i|
  puts i
end
```

**Advantages of Arrays in Ruby** - Dynamic size, Easy data storage, Supports multiple data types, Easy iteration

Ruby arrays provide flexible and efficient storage of multiple values along with various operations like insertion, deletion, traversal, and searching.

### **Q7b) What are the Positive Aspects of Rails? Explain with Example.**

**Rails** - Ruby on Rails is an open-source web application framework written in Ruby. It follows: MVC architecture, Convention over Configuration (CoC), DRY (Don't Repeat Yourself)

#### **Positive Aspects of Rails -**

**1) Faster Development** - Rails provides built-in tools and libraries that speed up development.

**2) MVC Architecture** - Separates: Model, View, Controller. This improves maintainability.

**3) Convention over Configuration** - Rails follows predefined conventions, reducing configuration effort.

**4) DRY Principle** - Avoids code duplication and improves code reuse.

**5) Database Integration** - Easy database connectivity using Active Record.

**6) Scaffolding Support** - Automatically generates: Models, Controllers, Views

**7) Security Features** - Provides built-in protection against: SQL Injection, Cross-site scripting

**8) Easy Maintenance** - Clean and organized structure simplifies maintenance.

#### **Example of Rails Controller**

```
class WelcomeController < ApplicationController
  def index
    @msg = "Welcome to Rails"
  end
end
```

**Example of Rails View** - <h2><%= @msg %></h2>

**Output** - Welcome to Rails

**Advantages of Rails** - Rapid web development, Better code organization, Reusable code, Easy testing, Open-source framework

**Applications of Rails** - E-commerce websites, Social networking sites, Enterprise web applications

Ruby on Rails simplifies web application development using MVC architecture, built-in tools, and rapid development features, making it efficient for modern web applications.

#### **Q8a) What is a Module? Explain the Difference Between Classes and Modules.**

**Module in Ruby** - A module in Ruby is a collection of: Methods, Constants, Classes

Modules are used to: Achieve code reusability, Provide namespace support, Implement mixins. Unlike classes, modules cannot create objects.

#### **Syntax of Module** -

```
module Demo
  def show
    puts "Inside Module"
  end
end
```

#### **Using Module** -

```
module Demo
  def show
    puts "Welcome"
  end
end
```

```
class Test
include Demo
end
t = Test.new
t.show
```

**Output** - Welcome

**Features of Module** - Cannot instantiate objects, Supports code reuse, Used as namespace, Supports multiple inheritance using mixins

| Basis            | Class                         | Module                        |
|------------------|-------------------------------|-------------------------------|
| Object Creation  | Objects can be created        | Objects cannot be created     |
| Constructor      | Supports constructor          | No constructor                |
| Inheritance      | Supports inheritance          | Cannot inherit directly       |
| Purpose          | Used for object creation      | Used for code sharing         |
| Instantiation    | Possible                      | Not possible                  |
| Method Inclusion | Not included in other classes | Can be included using include |
| Namespace        | Limited                       | Mainly used for namespace     |

### Example of Class

```
class Student
def display
puts "Student Class"
end
end
s = Student.new
s.display
```

**Output** - Student Class

**Advantages of Modules** - Reduces code duplication, Improves maintainability, Supports reusable programming

Classes are used to create objects and implement object-oriented programming, whereas modules are mainly used for code reuse and namespace management.

**Q8b) What is EJB? Draw and Explain Main Components of EJB Architecture.**

**EJB** stands for: Enterprise JavaBeans

EJB is a server-side component architecture used for developing enterprise-level distributed applications in Java.

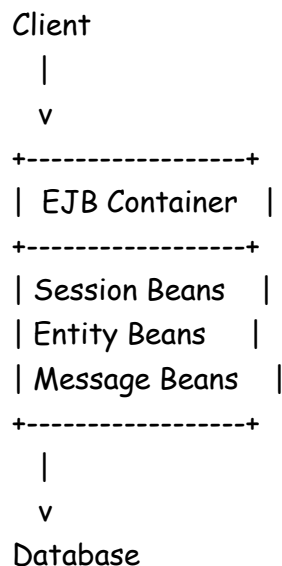
It simplifies: Transaction management, Security, Database operations, Distributed computing

EJB is a part of Java EE (Enterprise Edition).

**Features of EJB** - Scalability, Transaction management, Security support, Distributed application support, Reusable business components

**EJB Architecture** - EJB architecture consists of: Client, EJB Container, Enterprise Beans, Database

### **EJB Architecture Diagram**



### **Main Components of EJB Architecture**

**1) EJB Container** - EJB container provides runtime environment for Enterprise Beans.

**Functions of EJB Container** - Transaction management, Security, Lifecycle management, Resource pooling

**2) Enterprise Beans** - Enterprise Beans contain business logic. Types: Session Beans, Entity Beans, Message Driven Beans

**a) Session Beans** - Used for handling business logic. Types: Stateless, Stateful

**b) Entity Beans** - Represents persistent data stored in database.

**c) Message Driven Beans** - Used for asynchronous messaging.

**3) Client** - The client accesses EJB services. Client may be: Web application, Standalone application, Another enterprise bean

**4) Database** - Stores application data accessed by Enterprise Beans.

**Working of EJB Architecture -**

1. Client sends request
2. EJB container receives request
3. Container invokes enterprise bean
4. Bean processes business logic
5. Database interaction occurs
6. Response returned to client

**Advantages of EJB** - Simplifies enterprise application development, Provides security and transactions, Supports scalability, Reduces development complexity

**Applications of EJB** - Banking systems, E-commerce applications, Enterprise management systems

EJB is a powerful server-side architecture for developing scalable and distributed enterprise applications using reusable business components managed by EJB container.

**Q7a) Explain Ruby with Respect to Scalar Types & Their Operations, Control Statements & Arrays**

**Ruby** is an object-oriented, interpreted, and dynamic programming language used for web application development.

It is simple, flexible, and widely used with Rails framework.

**1) Scalar Types in Ruby** - Scalar types store single values.

Main scalar types in Ruby are: Integer, Float, String, Boolean

**a) Integer** - Stores whole numbers.

```
a = 10
puts a
```

**b) Float** - Stores decimal numbers.

```
b = 3.14
puts b
```

**c) String** - Stores sequence of characters.

```
name = "Rahul"
puts name
```

**d) Boolean** - Stores true or false.

```
flag = true  
puts flag
```

### **Operations on Scalar Types -**

#### **Arithmetic Operations -**

```
a = 10  
b = 5  
puts a + b  
puts a - b  
puts a * b  
puts a / b
```

**Output** - 15 5 50 2

#### **Comparison Operations -**

```
puts 10 > 5  
puts 10 == 5
```

**Output** - true false

#### **String Operations**

```
fname = "Hello"  
lname = "Ruby"  
puts fname + " " + lname
```

**Output** - Hello Ruby

**2) Control Statements in Ruby** - Control statements control the flow of execution.

### **Types of Control Statements**

1. Conditional Statements
2. Looping Statements

#### **a) if Statement -**

```
a = 10  
if a > 5  
puts "Greater"  
end
```

**Output** - Greater

#### **b) if-else Statement**

```
a = 3
if a > 5
  puts "Greater"
else
  puts "Smaller"
end
```

**Output - Smaller**

#### **c) case Statement**

```
day = 1
case day
when 1
  puts "Monday"
when 2
  puts "Tuesday"
else
  puts "Invalid"
end
```

#### **d) Loop Statements**

##### **for Loop**

```
for i in 1..5
  puts i
end
```

##### **while Loop**

```
i = 1
while i <= 3
  puts i
  i += 1
end
```

Ruby provides scalar data types, control structures, and arrays for efficient and flexible programming suitable for modern web applications.

#### **Q8) a) Explain Rails Applications & Databases. Also Explain Rails with AJAX**

**Rails Application** - Ruby on Rails is an open-source web framework written in Ruby.

Rails follows: MVC architecture, DRY principle, Convention over Configuration

**Rails Application Architecture** - User -> Controller -> Model -> Database -> View



## Components of Rails Application

### Component Purpose

Model      Handles database operations

View      User interface

Controller Handles requests

**Rails and Database** - Rails uses: Active Record for database interaction. Active Record maps database tables to Ruby objects.

**Supported Databases** - MySQL, SQLite, PostgreSQL, Oracle

**Example of Rails Model** - 

```
class Student < ApplicationRecord
  end
```

### Database Migration Example -

```
class CreateStudents < ActiveRecord::Migration[6.0]
  def change
    create_table :students do |t|
      t.string :name
      t.integer :roll
    end
  end
end
```

**Advantages of Rails Database Support** - Easy database handling, Automatic table mapping, Simplified CRUD operations

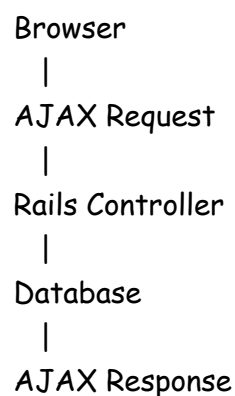
**Rails with AJAX** - AJAX stands for: Asynchronous JavaScript And XML. AJAX allows updating webpage content without reloading the complete page. Rails provides built-in AJAX support.

### Working of Rails with AJAX

1. User sends request
2. AJAX sends asynchronous request
3. Rails controller processes request

4. Partial response returned
5. Page updates dynamically

### Rails AJAX Architecture -



### Example of Rails AJAX - View File -

```
<%= button_to "Load",  
students_path,  
remote: true %>
```

### Controller -

```
def index  
  @students = Student.all  
end
```

**Advantages of Rails with AJAX** - Faster response, Better user experience, Partial page updates, Reduced server load

**Applications** - Dynamic websites, Real-time applications, Interactive web applications

Rails simplifies web application and database development using MVC and Active Record, while AJAX integration provides faster and interactive user experiences.

### Q7b) Explain Document Requests and Processing Forms in Rails

**Rails Request Processing** - In Ruby on Rails, a request is generated when a user interacts with a web application through browser. Rails processes requests using MVC architecture.

#### Request Processing Flow in Rails -

##### Steps in Request Processing

**1) User Sends Request** - User requests a webpage using URL.

Example: <http://localhost:3000/students>

**2) Router Receives Request** - Rails router checks routes file:

config/routes.rb

```
Example Route - Rails.application.routes.draw do  
  resources :students  
end
```

**3) Controller Processes Request**

Controller receives request and calls appropriate action.

**Example Controller** - class StudentsController < ApplicationController

```
def index  
  @students = Student.all  
end
```

User Request

|

v

Router

|

v

Controller

|

v

Model

|

v

View

|

v

Response to User

end

**4) Model Interacts with Database** - Model fetches or updates database records.

**Example Model** - 

```
class Student < ApplicationRecord
  end
```

**5) View Generates Response** - View displays data to user.

**Example View** - 

```
<h2>Student List</h2>
<%= @students.count %>
```

**Processing Forms in Rails** - Rails provides helper methods for form processing.

**Example Form** -

```
<%= form_with model: @student do |f| %>
Name :
<%= f.text_field :name %>
<br><br>
Roll :
<%= f.text_field :roll %>
<br><br>
<%= f.submit "Save" %>
<% end %>
```

### **Form Processing Steps**

1. User enters form data
2. Form submitted to controller
3. Controller receives parameters
4. Data stored in database
5. Response displayed

**Example Controller for Form Processing** -

```
def create
  @student = Student.new(student_params)
  @student.save
end
```

**Advantages of Rails Form Processing** - Automatic form generation, Simplified validation, Easy database integration

Rails efficiently handles document requests and form processing using MVC architecture and built-in helper methods.

### Q7a) Explain How Multiple Selection Constructs are Implemented in Ruby

Multiple selection constructs are used to execute different blocks of code based on multiple conditions.

Ruby mainly provides: if-elsif-else, case statement

**1) if-elsif-else Statement** - Used when multiple conditions are checked sequentially.

**Syntax -**

```
if condition1
statements
elsif condition2
statements
else
statements
end
```

**Example -**

```
marks = 75
if marks >= 80
puts "Distinction"
elsif marks >= 60
puts "First Class"
elsif marks >= 40
puts "Pass"
else
puts "Fail"
end
```

**Output -** First Class

**2) case Statement** - case statement is used as an alternative to multiple if-elsif conditions. It improves readability and efficiency.

**Syntax**

```
case variable
```

```
when value1
statements
```

```
when value2
statements
```

```
else
statements
```

```
end
```

**Example -** day = 2

```
case day
```

```
when 1
puts "Monday"
```

```
when 2
puts "Tuesday"
```

```
when 3
puts "Wednesday"
```

```
else
puts "Invalid Day"
```

```
end
```

**Output -** Tuesday

**Advantages of Multiple Selection Constructs** - Simplifies decision making ,Improves code readability, Reduces complexity, Easy maintenance

Ruby provides multiple selection constructs like if-elsif-else and case statements to implement decision-making efficiently and improve program readability.

**Q8) a) Explain How to Write the Methods and Call the Method in Ruby with Example**

**Methods in Ruby** - A method in Ruby is a block of reusable code used to perform a specific task.

Methods help: Reduce code duplication, Improve modularity, Simplify maintenance

**Syntax of Method** -

```
def method_name
  statements
end
```

**Example of Method without Parameters -**

```
def display
  puts "Welcome to Ruby"
end
display
```

**Output** - Welcome to Ruby

**Method with Parameters -**

```
def add(a,b)
  sum = a + b
  puts sum
end
add(10,20)
```

**Output** - 30

**Method Returning Value -**

```
def square(n)
  return n*n
end
puts square(5)
```

**Output** -

25

**Calling a Method** - A method is called using its name.

Example: method\_name or method\_name(arguments)

**Advantages of Methods** - Code reusability, Better readability, Easy debugging, Modular programming

Methods in Ruby provide reusable and organized programming structures that simplify software development.

### Q8b) What are the Differences Between JavaBeans and EJB?

| Basis                 | JavaBeans                        | EJB                                          |
|-----------------------|----------------------------------|----------------------------------------------|
| Definition            | Reusable Java software component | Enterprise-level server-side component       |
| Purpose               | Used for local applications      | Used for distributed enterprise applications |
| Execution Environment | Runs on JVM                      | Runs inside EJB Container                    |
| Complexity            | Simple                           | More complex                                 |
| Transaction Support   | Not supported                    | Supported                                    |
| Security              | Limited                          | Built-in security                            |
| Distributed Computing | Not supported                    | Supported                                    |
| Scalability           | Less scalable                    | Highly scalable                              |
| Database Support      | Basic support                    | Advanced enterprise database support         |
| Lifecycle Management  | Managed by application           | Managed by EJB container                     |
| Remote Access         | Not available                    | Available                                    |
| Example Usage         | GUI applications                 | Banking and enterprise systems               |

### Q7a) Explain Classes and Objects in Ruby with Appropriate Examples

**Class in Ruby** - A class in Ruby is a blueprint used to create objects. A class contains: Variables, Methods, Properties, Behaviors

#### Syntax of Class -

```
class ClassName
# methods and variables
end
```

**Object in Ruby** - An object is an instance of a class.

Objects are used to access class methods and properties.

**Creating Object** - `object_name = ClassName.new`

**Example of Class and Object -**

```
class Student
  def display
    puts "Welcome to Ruby"
  end
end
# Creating Object
s = Student.new
# Calling Method
s.display
```

**Output** - Welcome to Ruby

**Explanation -**

**Element Purpose**

`class`     Defines class

`initialize` Constructor method

`@id`       Instance variable

`new`       Creates object

`show`      Method

**Example with Variables and Methods -**

```
class Employee
  def initialize(id,name)
    @id = id
    @name = name
  end
  def show
    puts "Employee ID : #{@id}"
    puts "Employee Name : #{@name}"
  end
end
e1 = Employee.new(101,"Rahul")
e1.show
```

**Output** - Employee ID : 101

Employee Name : Rahul

**Features of Classes and Objects** - Encapsulation, Reusability, Modularity, Data abstraction

**Advantages** - Simplifies programming, Better code organization, Supports object-oriented programming

Classes and objects are fundamental concepts of Ruby that support object-oriented programming by organizing data and methods efficiently.

**Q7b) Introduce the Concept of Rails Application. Describe Layouts & Stylesheet in Rails**

**Layouts in Rails** - Layouts provide common webpage structure shared across multiple pages. Layouts reduce code duplication.

**Default Layout File** - `app/views/layouts/application.html.erb`

### **Example of Layout -**

```
<html>
<head>
<title>MyApp</title>
</head>
<body>
<h2>Welcome to Rails</h2>
<%= yield %>
</body>
</html>
```

**Explanation** - `<%= yield %>` - Inserts view content into layout

**Advantages of Layouts** - Common design for all pages, Reduces repeated code, Easy maintenance

**Stylesheet in Rails** - Stylesheets are used to define webpage appearance using CSS.

Rails stores stylesheets in: `app/assets/stylesheets`

### **Example of Stylesheet -**

```
Body {
  background-color: lightblue;
}
h2 {
  color: red;
}
```

**Including Stylesheet in Layout** - `<%= stylesheet_link_tag "application" %>`

**Advantages of Stylesheets** - Improves webpage appearance, Separates design from logic, Easy UI customization

**Applications of Rails** - E-commerce websites, Social networking applications, Enterprise web applications

Rails provides a powerful MVC-based framework for web development. Layouts ensure reusable page structure, while stylesheets improve webpage design and user interface consistency.